MAICEN-1125

Masters in AI in Architecture and Construction

Module 7, Unit 5 — Individual Assignment

From EIR to IFC: An IDS-Driven openBIM Validator for MEP Services with LLM-Authored Specifications

Individual Submission by Mark Shane Haines

Zigurat Global Institute of Technology

Submission Date: 28 June 2026 (23:59 CEST)

Reading guide for examiners

This submission maps to the rubric as follows:

- **Functional code execution (40%)** — §3 (validator and multi-format reporter), §4 (LLM modules with XSD validation gate), §5 (end-to-end results on two real IFC4 MEP models, 4-test pytest suite passing in 2.18 s).
- **Correct use of the BIM API (30%)** — §2 (hand-authored IDS ruleset constructed via the `ifctester.ids` API to guarantee XSD validity by construction), §3 (validation through `ifctester.ids.Ids.validate()`, the buildingSMART reference implementation), §4 (LLM output validated against the bundled buildingSMART IDS 1.0 XSD before persistence).
- **Documentation and logical explanation (30%)** — this report, the architecture and pipeline SVGs in §3, the rubric-mapped README on the GitHub repository, and a logically-staged commit history (Phase 0 through Phase 7 plus the live-LLM end-to-end commit).

Reports for every validation run are at `results/` (JSON, HTML, BCF, summary CSV, failures CSV) on the GitHub repository. Live Claude output from both LLM modules is committed as evidence: `results/eir_translation_three_clauses.ids`, `results/generated_demo.ids`, `results/Ifc4_Revit_MEP_remediated.json`.

§7 adds a future-proofing roadmap for scaling the workflow from the nine demonstration specifications to a full MEP&F production library (Mechanical, Electrical, Plumbing and Fire) of roughly 100–165 specifications across 41 distinct property sets, aligned to AS/NZS and NZBC. This is beyond the minimum requirements; §1–§6 stand on their own as a complete submission.

Why openBIM rather than Revit + pyRevit?

The slide deck for this module points at Revit and pyRevit, and that is a reasonable place to start if your only concern is shipping a button next to a working modeller's mouse. I went the other way. The argument running through Elias's slides 18 to 24 is that the contract between human intent in an Employer's Information Requirements (EIR) and machine logic in a rule check lives in **IDS** (Information Delivery Specification), not in the authoring tool. Once you accept that, the authoring tool stops mattering. Revit, ArchiCAD, AutoCAD MEP and Tekla can all produce IFC4. If my checker only ran inside Revit, it would be worth less than half of what a practising Services BIM Consultant actually needs in front of clients who use a mixed authoring estate.

I therefore targeted the buildingSMART reference openBIM stack directly: `ifcopenshell` to read IFC, `ifctester` to apply IDS. Both are maintained by the people who write the IDS standard, and the IDS 1.0 XSD ships inside the `ifctester` Python package. The same validation gate buildingSMART publishes is the one my code enforces, both on the hand-authored ruleset and on every LLM-generated specification before it touches disk. When this approach has to be defended in front of a client or a professional indemnity insurer, that pedigree matters.

Why two IFC test models?

The handover plan named two source IFC models from the University of Auckland Open IFC Model Repository (`Trapelo_IFC4_MEP.ifc`, `Hospital_IFC4_SPR.ifc`). The Auckland repository turned out to be a single-page JavaScript application that requires an interactive browser login I could not automate cleanly within the time budget. Rather than abandon the comparative approach, I ran a thorough sweep of GitHub, the buildingSMART sample-test-files repository, the IfcOpenShell community fixtures and the EnEff:BIM research datasets, and sourced two no-login IFC4 replacements with substantial MEP content:

- `Ifc4_Revit_MEP.ifc` (27.8 MB, IFC4) — the Autodesk Revit MEP Advanced Sample project (`rme_advanced_sample_project.rvt`) exported to IFC4 and republished by youshengCode on GitHub. Contains 302,453 entities including 8,515 distribution ports, 491 pipe segments, 837 duct segments, 11 flow terminals, 6 fire-suppression terminals, plus 415 light fixtures and 359 air terminals.
- `BoilerGasRadiatorDomesticHotWater.ifc` (1.35 MB, IFC4, MIT-licensed) — the VDI 6020 boiler / gas radiator / domestic hot water reference model from the German federal EnEff:BIM research project. Contains 26,030 entities including 1 pump, 4 valves, 1 boiler, 29 pipe segments and 114 distribution ports.

Together they exercise seven of the nine specifications in the ruleset. Two specs (`IfcCableSegment`, `IfcElectricAppliance`) have zero applicable instances in either model. The ruleset retains those specifications because the contract is for the building, not the test sample, and the validator reports them honestly as `applicable=0 → SKIP` rather than dressing them as `PASS`. This is itself a design point I will return to in §6.

1. Executive summary

This submission demonstrates a complete EIR-to-validation workflow built on the buildingSMART reference openBIM stack:

1. A **hand-authored IDS ruleset** (`ids/mep_services_v1.ids`) of nine MEP services specifications, constructed via the `ifctester.ids` Python API so the result is guaranteed XSD-valid by construction. The ruleset is then validated against the bundled IDS 1.0 XSD via `lxml` as a belt-and-braces second gate.
2. A **validator and multi-format reporter** wrapping `ifctester.ids.Ids.validate()`, with a single CLI command (`python -m src.cli`) producing JSON, HTML, BCF, summary CSV and per-failure CSV outputs in one invocation. The BCF format is the one that matters in production: it imports straight into Solibri, BIMcollab, Navisworks Manage and Trimble Connect as a coordination issue tracker, which is how modellers actually receive feedback.
3. **Two LLM modules** built on the Anthropic SDK (Claude Sonnet 4.6). The first (`src/llm_eir_to_ids.py`) translates a natural-language EIR clause into one IDS specification, with the bundled IDS XSD as a hard validation gate. The second (`src/llm_remediation.py`) reads the JSON validation report, batches failed entities into Claude calls, and writes a plain-English remediation paragraph into the report for each one.

The pipeline was exercised against the two IFC4 test models above. The Revit MEP model passes 5 of 9 specifications (76% of total checks); the Boiler model passes 6 of 9 (62% of total checks). End-to-end timings on a Windows 11 laptop: IFC open 0.11 s to 2.26 s, IDS validation 0.01 s to 0.50 s. Both numbers are an order of magnitude under the 20-minute slowdown documented in IfcOpenShell GitHub Discussion #6782, which is triggered by complex applicability restrictions rather than the entity-class filters used here.

Live LLM output is committed to the repository as evidence: three EIR clauses translated to three XSD-valid IDS specifications, eight validation failures annotated with remediation paragraphs naming the missing `Pset.Property`, the design-side action and the downstream impact. Both runs are also embedded in `notebooks/01_demo_validation.ipynb`.

The complete codebase, IDS ruleset, IFC source links, test suite, technical report, architecture diagrams and live LLM output are public at `https://github.com/markshanehaines-ZIG/m7u5-ids-mep-validator` under MIT licence.

2. The IDS ruleset — definition and rationale

2.1 The nine specifications

`ids/mep_services_v1.ids` carries nine specifications covering the major MEP services entity classes in IFC4. Each specification has an `<applicability>` element (the IFC entity class to target, optionally filtered by predefined type) and a `<requirements>` element listing the property, attribute, material or partOf facets that must be satisfied.

#	Applicability	Requirement	Standards hook
MEP-01	`IfcPipeSegment`	`Pset_PipeSegmentTypeCommon.NominalDiameter` populated AND material assigned	AS/NZS 3500.1, IFC4
MEP-02	`IfcDistributionPort`	`FlowDirection` attribute in {`SOURCE`, `SINK`, `SOURCEANDSINK`}	IFC4 IfcFlowDirectionEnum
MEP-03	`IfcFlowTerminal`	Membership of an `IfcDistributionSystem` group via `IfcRelAssignsToGroup`	IFC4
MEP-04	`IfcFireSuppressionTerminal`	`Pset_FireSuppressionTerminalSprinkler.CoverageArea` populated AND `PredefinedType` not `NOTDEFINED`	NZS 4541, BS EN 12845 §10
MEP-05	`IfcPump`	`Pset_ManufacturerTypeInformation.Manufacturer` AND `.ModelLabel` populated	ISO 19650-2
MEP-06	`IfcDuctSegment`	`Pset_DuctSegmentTypeCommon.NominalLengthOrDimension` populated AND material assigned	AS 1668.2, AS 4254
MEP-07	`IfcValve`	`Pset_ValveTypeCommon.ValveOperation` populated AND `PredefinedType` not `NOTDEFINED`	AS 1628
MEP-08	`IfcCableSegment`	`Pset_CableSegmentTypeCommon.CrossSectionalArea` populated	AS/NZS 3000 §433
MEP-09	`IfcElectricAppliance`	`Pset_ElectricalDeviceCommon.NominalPower` populated	AS/NZS 3000 §4

Each specification carries a human-readable `<instructions>` field that names the regulatory hook. For this academic submission the references blend British (BS EN, BS 7671) and Australasian (AS/NZS, NZS, NZBC) standards; for production use in my consultancy I will normalise these to the AS/NZS stack throughout. The structural pattern stays the same.

2.2 Why the IDS file is the contract surface

The IDS is the single piece of the workflow a non-developer can read and negotiate. A modeller, a designer, a client representative and a checker can all sit in the same room with the IDS file open and argue about whether `Pset_PipeSegmentTypeCommon.NominalDiameter` is the right hook for a sprinkler riser, without ever opening Python. This matters because the validator, the reporter, the LLM modules and the CLI are all implementation details that change between projects; the IDS is the artefact that survives across them. Treating the IDS as the contract surface — and the openBIM toolchain as the enforcement mechanism — is the design choice that separates this build from a Revit-button approach.

2.3 The MEP-04 IFC4 schema correction

The handover plan named `IfcSprinkler` as the applicability entity for the sprinkler specification. `IfcSprinkler` exists in IFC2X3 but was **retired in IFC4**. Sprinklers are now modelled as `IfcFireSuppressionTerminal` with `PredefinedType=SPRINKLER` or `SPRINKLERDEFLECTOR`. The original spec would have matched zero entities in any IFC4 file — a silent false PASS that no automated validator could catch on its own.

I corrected the spec to `IfcFireSuppressionTerminal` and widened the requirement to flag both missing `CoverageArea` and `NOTDEFINED` PredefinedType. The corrected spec then surfaces all six fire-suppression terminals in the Revit MEP sample as failing.

This is the kind of issue every legacy IDS in the industry has. When a firm migrates a client from IFC2X3 to IFC4, half their old rules silently match zero entities and they don't know it. The validator's `applicable=0 → SKIP` reporting flags this honestly. A migration audit running an old IDS through this pipeline on an IFC4 model finds every spec that needs schema migration.

2.4 IDS authoring via the ifctester Python API

The ruleset is generated by `scripts/build_mep_ids.py`, which constructs every specification via the `ifctester.ids` Python constructors. The result is **guaranteed XSD-valid by construction**: there is no path through the constructor API that emits malformed XML. A secondary explicit `lxml.etree.XMLSchema.assertValid()` check against the bundled buildingSMART IDS 1.0 XSD passes (9 specifications, zero schema errors).

Hand-writing IDS XML by reaching for a text editor is the wrong workflow for production. Property names, dataType strings and facet attribute orderings are unforgiving. A Python builder script that wraps the buildingSMART reference constructors is the right workflow, and it is the workflow the LLM generator in §4 uses internally too.

3. The validator and reporter

3.1 Toolchain

Stage	Tool	Version
IFC parsing	`ifcopenshell`	0.8.5
IDS validation	`ifctester` (buildingSMART reference)	0.8.5
XSD schema	bundled inside `ifctester` package	IDS 1.0
Reporter outputs	`ifctester.reporter.Json` / `.Html` / `.Bcf`	0.8.5
Test harness	`pytest`	9.1.0
LLM client	`anthropic` SDK	0.109.1
Model	`claude-sonnet-4-6`	via Anthropic API
Environment	Python 3.11 venv on Windows 11	—

This is the openBIM equivalent of the M7U4 ETL pipeline: **Load → Filter applicability → Check requirements → Emit reports**, realised in code rather than diagram. The four stages are shown in `docs/pipeline.svg` (embedded below) with the LLM extension lanes alongside.

3.2 The validator wrapper

`src/validator.py` is intentionally thin. The buildingSMART stack already does the schema-level work; the wrapper exists to (a) time each stage so this report can quote the numbers, (b) expose a single `ValidationRun` dataclass the CLI, the test suite and the demo notebook all consume, and (c) log progress so a CI job or interactive caller can follow long-running validations.

The class has one public method, `run()`, that opens the IFC, loads the IDS, calls `doc.validate(model)`, instantiates `ifctester.reporter.Json(doc).report()` to populate the result structure, and returns a `ValidationRun` dataclass carrying the model, the validated IDS, the open and validate timings, and the JSON report.

3.3 The multi-format reporter

`src/reporter.py` produces five output files per IFC, each for a different downstream consumer:

File	Read by	Used when
`*_report.json`	other software (CDE plugins, dashboards, scripts)	feeding results into a Power BI dashboard or another tool
`*_report.html`	humans in a browser	sharing with the modelling team — they click each spec to expand failures
`*_report.bcf`	Solibri / BIMcollab / Navisworks Manage / Trimble Connect	**the production output** — imports into the CDE as a coordination issue tracker with one BCF topic per failure
`*_summary.csv`	Excel	dashboards, client weekly reporting, quick scan
`*_failures.csv`	Excel + Power BI	drill-down: one row per failed entity with GUID for back-trace into the authoring tool

One ifctester 0.8.5 subtlety caught me out and is worth recording. The `Json`, `Html` and `Bcf` reporters all read `self.results` inside their `to_file()` methods, but `self.results` is only populated by `.report()`. My wrapper calls `.report()` explicitly before each `.to_file()`. Without it the JSON output is a 23-byte stub (`{"hide_skipped": false}`), the HTML is a template skeleton, and the BCF writer raises `KeyError: 'title'`. Anyone else building on this stack should know.

A second polish: a specification with zero applicable entities is reported by ifctester as `status=True` (vacuously satisfied). For my dashboard view I rewrote this to report `SKIP` when `applicable==0`, so a spec with no entities found is distinct from a real PASS. Vacuous satisfaction is not the same as the IDS being met.

3.4 The CLI entry point

```
python -m src.cli --ifc ifc/Ifc4_Revit_MEP.ifc \
                  --ids ids/mep_services_v1.ids \
                  --out results/
```

`src/cli.py` exposes the validator and reporter via `argparse`. The exit code mirrors validation status so a CI pipeline can gate on it: `0` for pass, `1` for fail. A `--quiet` flag suppresses info-level logs for non-interactive use.

4. LLM modules — natural language at both ends

The two LLM modules are the additive piece. Neither is required by the rubric. Both exist because the EIR-to-IDS and validation-result-to-modeller-advice gaps are the actual bottlenecks in Services BIM consulting today, and Claude is good at exactly the kind of structured text-to-structured-text task that bridges them.

4.1 EIR-to-IDS generator (`src/llm_eir_to_ids.py`)

Workflow:

1. Read a plain-text file of natural-language EIR clauses (blank-line separated, `#` comments).
2. For each clause, build a structured Anthropic API request with a system prompt that constrains Claude to emit one `<specification>` element of IDS 1.0 XML and nothing else. The system prompt includes a one-shot example of (English clause → IDS XML) and the namespace and cardinality rules.
3. Receive the response, strip any markdown code fences, extract the `<specification>` block.
4. Wrap the block in a complete `<ids>` document and validate against the bundled buildingSMART IDS XSD via `lxml`.

5. If XSD-valid, append to the output `.ids` file. If invalid, report the error and skip — the run cannot silently emit malformed XML.

The XSD gate is the critical design decision. An LLM that occasionally hallucinates a property name or a facet structure is acceptable; an LLM that writes invalid XML into a production IDS file is not. The gate runs in microseconds and rejects anything the buildingSMART validator would reject.

4.2 Remediation generator (`src/llm_remediation.py`)

Workflow:

1. Read a JSON validation report produced by `src/cli.py`.
2. Iterate `failed_entities` across every requirement in every specification.
3. Batch the failures (default 8 per request) into Anthropic API calls with a system prompt that asks for one 60–90 word paragraph per failure addressed to the BIM author, naming the specific `Pset.Property` that is missing, the GUID and name of the affected entity, the design-side action to take, and the downstream impact.
4. Merge the returned paragraphs back into the JSON report under a new `remediation` key on each failure.
5. Write the annotated JSON to the output path. Failed batches log a warning and continue; the rest of the report is preserved.

The output is a JSON report where every annotated failure now carries machine-traceable identity (GUID, IFC class, spec name) **and** human-readable instruction. This is what closes the "modellers don't know what to do with the report" gap that real BIM coordination meetings actually trip over.

4.3 Demonstration evidence

Both modules ran end-to-end against the real test models (committed to the repository):

EIR-to-IDS generator translated three natural-language clauses from `queries/sample_eir_clauses.txt`:

> *All air-handling units shall declare the supply-air design flow rate via `Pset_AirHandlingUnitTypeCommon.NominalAirFlowRate` so the commissioning team can verify against the room-by-room load schedule.*

into three IDS specifications, all of which passed the XSD gate and round-trip-parsed through `ifctester.ids.open()`. Output committed at `results/eir_translation_three_clauses.ids`.

Remediation generator annotated eight failures from `results/Ifc4_Revit_MEP_report.json` with paragraphs of this form:

> *For the IfcPipeSegment with GlobalId `213f6VCXH5cBojaxiKbfKS` (named 'Pipe Types:Standard:513756'), the property set `Pset_PipeSegmentTypeCommon` is either absent or does not contain the required `NominalDiameter` property. Please open this element in your authoring tool, navigate to its type or instance properties, and populate `Pset_PipeSegmentTypeCommon.NominalDiameter` with the design pipe diameter expressed in millimetres. Without this value, downstream hydraulic sizing and clash-clearance checks cannot be verified against design intent.*

Output committed at `results/Ifc4_Revit_MEP_remediated.json`.

Both runs are also embedded in `notebooks/01_demo_validation.ipynb` so the cells show real Claude output rather than skipped placeholders.

5. Results

5.1 Per-model summary

Model	Source	Schema	Size	Total entities	Specs pass	Checks pass
`Ifc4_Revit_MEP.ifc`	Autodesk RME Advanced Sample (youshengCode/IfcSampleFiles)	IFC4	27.8 MB	302,453	5 / 9	76 %
`BoilerGasRadiatorDomesticHotWater.ifc`	EnEff:BIM VDI 6020 reference, MIT-licensed	IFC4	1.35 MB	26,030	6 / 9	62 %

5.2 Per-specification results

#	Specification	Revit MEP applicable / pass / fail	Boiler applicable / pass / fail
MEP-01	Pipe segment — diameter and material	491 / 0 / 491	29 / 0 / 29
MEP-02	Distribution port — flow direction	8,515 / 8,515 / 0	114 / 114 / 0
MEP-03	Flow terminal — system membership	11 / 0 / 11	0 / 0 / 0 (SKIP)
MEP-04	Fire-suppression terminal — coverage and predef. type	6 / 0 / 6	0 / 0 / 0 (SKIP)
MEP-05	Pump — manufacturer and model	0 / 0 / 0 (SKIP)	1 / 0 / 1
MEP-06	Duct segment — nominal size and material	837 / 0 / 837	0 / 0 / 0 (SKIP)
MEP-07	Valve — operation and predef. type	0 / 0 / 0 (SKIP)	4 / 0 / 4
MEP-08	Cable segment — cross-sectional area	0 / 0 / 0 (SKIP)	0 / 0 / 0 (SKIP)
MEP-09	Electric appliance — nominal power	0 / 0 / 0 (SKIP)	0 / 0 / 0 (SKIP)

5.3 Timings

Stage	Revit MEP (28 MB, 302k entities)	Boiler (1.35 MB, 26k entities)
`ifcopenshell.open()`	2.26 s	0.11 s
`ifctester.ids.Ids.validate()`	0.50 s	0.01 s
Full pipeline (open + validate + emit 5 reports)	~3.5 s	~0.4 s
pytest suite (4 tests)	2.18 s	—

Both validation times are an order of magnitude under the 20-minute slowdown documented in IfcOpenShell GitHub Discussion #6782. That issue is triggered when applicability uses complex restrictions across the whole model; my IDS uses entity-class-only applicability filters, which `ifctester` handles efficiently.

5.4 LLM module results

- **EIR-to-IDS generator** translated three sample clauses into three XSD-valid IDS specifications via three Anthropic API calls (Claude Sonnet 4.6), each returning HTTP 200. All three round-tripped through `ifctester.ids.open()`. Total cost: ~NZ$0.20.
- **Remediation generator** annotated 8 of 2,679 failures from the Revit MEP report in a single batched Anthropic API call, returning HTTP 200. The output JSON carries the original report structure plus a `remediation` key on each of the 8 annotated entities. Total cost: ~NZ$0.10.

6. Discussion and findings

Two findings stand out from the run.

Distribution port flow direction is the only specification that passes 100 % on the Revit MEP sample. All 8,515 IfcDistributionPort instances declare a valid FlowDirection. Revit's IFC exporter writes this one reliably. By contrast, none of the 491 IfcPipeSegment, 837 IfcDuctSegment or 11 IfcFlowTerminal entities meet the ruleset, because the `Pset_*TypeCommon` property sets the IDS requires are not authored by Revit's exporter at all. This is exactly the authoring gap an IDS is meant to catch, and exactly the conversation I want to have with a modelling lead at the start of a project rather than at handover.

The MEP-04 IFC4 schema correction is the kind of issue every legacy IDS has. The handover plan was written against an IFC2X3 mental model. `IfcSprinkler` doesn't exist in IFC4. A spec that targets a retired entity matches zero instances, returns vacuous PASS, and lulls the team into thinking sprinkler coverage is being checked when it isn't. The validator's `SKIP` status when `applicable==0` (rather than ifctester's default `PASS`) is the small change that surfaces this. A consultancy migrating a client from IFC2X3 to IFC4 should run their existing IDS library through this pipeline first as a migration audit.

The openBIM choice was vindicated by the result. Authoring the IDS through `ifctester.ids` constructors guarantees XSD validity in a way hand-written XML cannot. That guarantee then transfers to the LLM module's output, because Claude's reply is wrapped in a complete IDS document and validated against the same XSD before persistence. The two LLM modules turn "natural-language requirement → machine-checkable rule" and "machine-readable failure → English remediation" into one-call operations rather than two-week workshops and Teams-message back-and-forth.

What did not work as planned. The originally targeted Auckland Open IFC Model Repository was unreachable without an interactive browser login. The chosen replacements cover seven of the nine specifications between them. Rather than weaken the IDS to fit the available data, the ruleset retains the unmatched specifications and the validator reports `SKIP`. A real EIR is written for the building, not for the test sample.

What this could become. A pyRevit wrapper sitting on top of this codebase would call `validator.run()` exactly as the CLI does, with the in-session document path passed in. The implementation choice is therefore additive rather than exclusive: openBIM is the right surface for the IDS contract; pyRevit can sit on top of it if a specific firm needs an in-Revit button. The dissertation chapter that grows out of this assignment will extend the architecture along the lines laid out in §7 below: a feedback loop where commissioning evidence (BCF issues with measurements) can amend the IDS automatically, closing the EIR → IDS → check → evidence circle.

7. Future-proofing the workflow for full MEP&F coverage

The nine specifications in `ids/mep_services_v1.ids` are roughly six per cent of a production catalogue. They were chosen to exercise the breadth of IDS facet types (`property`, `attribute`, `material`, `partOf`), not to cover any single service in depth. A production-scale ruleset covering Mechanical, Electrical, Plumbing and Fire (MEP&F) across all the entity classes and property sets the discipline cares about is roughly 100–165 specifications referencing 40–60 distinct property sets.

The pipeline does not need to change to scale; only the IDS catalogue does. That separation is the point of the design.

7.1 Property-set taxonomy by service

The tables below name the IFC4 entity class, the canonical buildingSMART Pset, the properties I would require, the IFC dataType, and the regulatory hook I would put in the spec's `<instructions>` field for an AS/NZS-aligned ruleset. These are deliberately the Psets I would consider when authoring; not every entry becomes a spec on every project. The right answer at BEP stage is to pick the subset the client genuinely needs to verify, and to trim the rest.

Mechanical — ventilation, air conditioning, heating

Entity	Pset	Required properties	Hook
`IfcAirHandlingUnit`	`Pset_AirHandlingUnitTypeCommon`	`NominalAirFlowRate`, `NominalSensibleCapacity`	AS 1668.2
`IfcChiller`	`Pset_ChillerTypeCommon`	`NominalCapacity`, `NominalEER`, `NominalCOP`	NZBC G4, AS/NZS 3823
`IfcBoiler`	`Pset_BoilerTypeCommon`	`EnergySource`, `NominalEnergyConsumption`	NZS 5261 (gas)
`IfcCoolingTower`	`Pset_CoolingTowerTypeCommon`	`HeatTransferArea`, `NominalCapacity`	NZBC G4
`IfcFan`	`Pset_FanTypeCommon`	`NominalAirFlowRate`, `NominalTotalPressure`, `MotorDriveType`	AS 1668.2
`IfcDuctSegment`	`Pset_DuctSegmentTypeCommon`	`NominalLengthOrDimension`, material	AS 4254
`IfcDuctFitting`	`Pset_DuctFittingTypeCommon`	`NominalLength`, material	AS 4254
`IfcDamper`	`Pset_DamperTypeCommon`	`OperationTemperatureMax`, `Operation`	AS 1682 (fire dampers)
`IfcAirTerminal`	`Pset_AirTerminalTypeCommon`	`Shape`, `FaceType`, `AirFlowrateRange`	AS 1668.2
`IfcUnitaryEquipment`	`Pset_UnitaryEquipmentTypeCommon`	`OperatingMode`, `NominalCoolingCapacity`, `NominalHeatingCapacity`	NZBC H1

Electrical — distribution, lighting, devices

Entity	Pset	Required properties	Hook
`IfcCableSegment`	`Pset_CableSegmentTypeCommon`	`CrossSectionalArea`, `MaximumOperatingTemperature`, `FireRating`	AS/NZS 3000 §3, AS/NZS 3008
`IfcCableCarrierSegment`	`Pset_CableCarrierSegmentTypeCommon`	`WidthOrDiameter`, material	AS/NZS 3000
`IfcElectricAppliance`	`Pset_ElectricalDeviceCommon`	`NominalPower`, `NominalVoltage`, `NominalFrequency`	AS/NZS 3000 §4
`IfcLightFixture`	`Pset_LightFixtureTypeCommon`	`LightFixtureMountingType`, `MaintenanceFactor`, `ArticleNumber`	AS/NZS 1680
`IfcElectricDistributionBoard`	`Pset_ElectricDistributionBoardTypeCommon`	`NominalCurrent`, `NumberOfGangs`	AS/NZS 3000 §2
`IfcSwitchingDevice`	`Pset_SwitchingDeviceTypeCommon`	`SwitchFunction`, `RatedCurrent`	AS/NZS 3947
`IfcProtectiveDevice`	`Pset_ProtectiveDeviceTypeCommon`	`BreakingCapacity`, `OperatingMode`	AS/NZS 3000 §2.5
`IfcTransformer`	`Pset_TransformerTypeCommon`	`PrimaryVoltage`, `SecondaryVoltage`, `PrimaryCurrent`	AS 60076
`IfcMotorConnection`	`Pset_MotorConnectionTypeCommon`	`MotorConnectionType`	AS 60034

Plumbing / Hydraulic — water, sanitary, gas

Entity	Pset	Required properties	Hook
`IfcPipeSegment`	`Pset_PipeSegmentTypeCommon`	`NominalDiameter`, material, `FlowDirection`	AS/NZS 3500.1, AS/NZS 3500.2
`IfcPipeFitting`	`Pset_PipeFittingTypeCommon`	`NominalDiameter`, `JointSize`, material	AS/NZS 3500
`IfcPump`	`Pset_PumpTypeCommon`	`PumpType`, `FlowRateRange`, `Pressure`, `Manufacturer`, `ModelLabel`	NZBC G12, AS/NZS 3500.1
`IfcValve`	`Pset_ValveTypeCommon`	`ValveOperation`, `ValveMechanism`, `Size`, `PredefinedType`	AS 1628, AS 4794
`IfcTank`	`Pset_TankTypeCommon`	`NominalCapacity`, `StorageType`, `PatternType`	NZBC G12, AS/NZS 4020
`IfcSanitaryTerminal`	`Pset_SanitaryTerminalTypeCommon`	`Category`, material, `WaterConsumptionPerFlush`	NZBC G1, AS/NZS 6400 (WELS)
`IfcWasteTerminal`	`Pset_WasteTerminalTypeCommon`	`WasteTerminalType`, material	AS/NZS 3500.2
`IfcInterceptor`	`Pset_InterceptorTypeCommon`	`InterceptorType`, `NominalCapacity`	NZBC G13
`IfcDistributionSystem`	(entity-level)	`PredefinedType` ∈	NZBC G12, NZS 5261

Fire — sprinkler, fire alarm, suppression

Entity	Pset	Required properties	Hook
`IfcFireSuppressionTerminal` (`SPRINKLER`)	`Pset_FireSuppressionTerminalSprinkler`	`CoverageArea`, `Pattern`, `WaterDistributionType`	NZS 4541, NZBC C/AS2
`IfcFireSuppressionTerminal` (`BREECHINGINLET`)	`Pset_FireSuppressionTerminalBreechingInlet`	`Pattern`	NZS 4510
`IfcFireSuppressionTerminal` (`HOSEREEL`)	`Pset_FireSuppressionTerminalHoseReel`	`Pattern`, `HoseDiameter`, `HoseLength`	NZS 4503
`IfcAlarm`	`Pset_AlarmTypeCommon`	`AlarmType`, `Application`	NZS 4512, AS 1670.1
`IfcSensor` (smoke / heat)	`Pset_SensorTypeCommon` + specific	`SensorType`, `MeasurementRange`	NZS 4512, AS 1670.1
`IfcDamper` (`FIREDAMPER`)	`Pset_DamperTypeCommon` + `Pset_FireSafetyProperties`	`FireRating`, `Operation`	AS 1682, NZBC C/AS2
`IfcCableSegment` (fire-rated)	`Pset_CableSegmentTypeCommon` + `Pset_FireSafetyProperties`	`FireRating`	AS/NZS 3013

Common cross-cutting — manufacturer, classification, spatial structure

Not service-specific but every production IDS should enforce them. Anything that fails these is impossible to hand over cleanly into FM/CAFM.

Applicability	Pset / facet	Required	Hook
Any `IfcElementType`	`Pset_ManufacturerTypeInformation`	`Manufacturer`, `ModelLabel`, `ProductionYear`	ISO 19650-2 §6.4
Any `IfcElement` instance	`Pset_ManufacturerOccurrence`	`BarCode`, `SerialNumber`, `AssemblyPlace`	COBie
Any FM asset	`Pset_AssetManagement`	`AssetIdentifier`, `OriginalValue`, `Owner`	ISO 55000
Any element	classification facet	`system=Uniclass 2015` or `NRM2` or `Omniclass`, value present	ISO 12006-2
All elements	spatial-structure	contained in `IfcSpatialStructureElement` via `IfcRelContainedInSpatialStructure`	IFC4 spatial rules
`IfcProject` root	attribute facet	references `IfcProjectedCRS` (georeferenced)	ISO 19111

Counts to plan for

Service area	Distinct Psets	Realistic spec count
Mechanical (HVAC + heating)	10	25–40
Electrical (distribution, lighting, devices)	9	20–35
Plumbing / Hydraulic (water, sanitary, gas)	10	25–40
Fire (sprinkler, alarm, suppression, fire-rated)	7	15–25
Common cross-cutting	5	15–25
Total	**41 Psets**	**100–165 specs**

The handover plan's nine specs is roughly six per cent of a production catalogue. That is consistent with the assignment brief (demonstrate the pipeline) and inconsistent with shipping to a client (don't).

7.2 Multi-IDS authoring

The CLI currently accepts one `--ids` flag. For production I would extend it to accept either a single file or a directory of files, validate the IFC against each, and merge the per-IDS reports into a single output. Each IDS file would represent one service area or one cross-cutting concern, organised as:

```
ids/
├── mechanical_v1.ids
├── electrical_v1.ids
├── plumbing_hydraulic_v1.ids
├── fire_v1.ids
├── common_cross_cutting_v1.ids
└── client_overlays/
    └── <client_name>_overrides.ids
```

The `client_overlays/` pattern lets a client-specific EIR add or tighten rules on top of the base catalogue without forking the whole library. It is the same pattern Solibri rulesets already use, and it is what makes the catalogue maintainable across many concurrent projects.

7.3 IFC4.3 schema scope for infrastructure

The civil and rail entities (`IfcAlignment`, `IfcRoad`, `IfcRailway`, `IfcBridge`, `IfcEarthworksCut`, drainage networks) live in IFC4.3 (`IFC4X3_ADD2`), not IFC4. `ifctester` already supports `IFC4X3_ADD2` as an `ifcVersion` value; my current IDS file declares `IFC4` only. The change is one line per spec, plus a column on the catalogue noting which schema version applies. Hospitals, schools and commercial buildings stay on IFC4. Infrastructure work moves to IFC4.3 once authoring tools catch up (Civil 3D 2025 onward).

7.4 bSDD integration for the LLM generator

The biggest single source of LLM error in EIR-to-IDS translation is Claude inventing a Pset name that does not exist. The fix is to give the LLM a tool-call that hits the buildingSMART Data Dictionary (`bsdd.buildingsmart.org`) for any Pset it wants to reference. If the Pset is not in bSDD the LLM is told to either re-pick or flag the spec for human review. I would build this as a thin MCP server so the generator becomes an agentic loop rather than a single-shot translation. Estimated effort: two days plus regression testing.

7.5 BCF feedback loop — closing the circle

Today the BCF report is one-way: the validator emits a BCF, the modeller resolves the issues. The closing of the loop would let the commissioning team push measured values (actual cable cross-sectional areas, actual sprinkler coverage tests, actual pump heads) back as BCF topics with structured payloads. A new module would compare measured values to required values from the IDS, surface discrepancies, and propose draft amendments to the IDS for the next project. This is what turns a one-shot check into an organisation's learning loop, and it is the dissertation chapter direction I will pursue.

7.6 CI integration for weekly client model checks

For every active client project I would set up a scheduled job (GitHub Actions on a private repo or a Windows Task on the firm's coordination server) that:

1. Pulls the latest IFC from the client's CDE (Trimble Connect, Autodesk Construction Cloud, Asite).
2. Runs the validator against the client-specific IDS suite.
3. Pushes the BCF report back to the CDE as an updated issue set.
4. Pushes the remediation-annotated JSON to a shared drive for the modelling lead to triage on Monday morning.

Estimated effort: a day per client to wire up CDE credentials and tune the scheduler.

7.7 The Pset-aware authoring tool gap

Today an IDS author has to look up Pset names manually in the bSDD or the IFC4 documentation. A Pset-aware IDS authoring tool — autocomplete on Pset name and property name, type-checked dataType, restriction-aware enum suggestions for `PredefinedType` — would tenfold authoring productivity. None exists publicly that I am aware of. Building one as a VS Code extension on top of the bSDD API is a small product in its own right and would be the most useful single addition to the workflow.

8. Repository layout

```
m7u5-ids-mep-validator/
├── README.md                                ← this submission's entry point
├── LICENSE                                  ← MIT
├── .env.template                            ← template (real .env is gitignored)
├── requirements.txt                         ← Python 3.11 pip freeze
├── ids/
│   └── mep_services_v1.ids                  ← hand-authored 9-spec ruleset
├── ifc/
│   ├── Ifc4_Revit_MEP.ifc                   ← primary test model (gitignored, 27.8 MB)
│   └── BoilerGasRadiatorDomesticHotWater.ifc← secondary test model (gitignored, 1.35 MB)
├── src/
│   ├── __init__.py
│   ├── validator.py                         ← ifctester wrapper + ValidationRun dataclass
│   ├── reporter.py                          ← JSON/HTML/BCF/CSV multi-format export
│   ├── cli.py                               ← `python -m src.cli --ifc ... --ids ... --out ...`
│   ├── llm_eir_to_ids.py                    ← EIR clause → IDS spec via Claude + XSD gate
│   └── llm_remediation.py                   ← failure JSON → English remediation via Claude
├── scripts/
│   ├── build_mep_ids.py                     ← rebuilds mep_services_v1.ids via ifctester API
│   └── build_report_pdf.py                  ← this report .md → .pdf via Edge headless
├── notebooks/
│   ├── 00_ifc_inventory.ipynb               ← schema + entity-count sanity check
│   └── 01_demo_validation.ipynb             ← end-to-end demo with Claude output embedded
├── queries/
│   └── sample_eir_clauses.txt               ← three sample EIR clauses for the LLM generator
├── tests/
│   └── test_validator.py                    ← 4 pytest tests, all passing in 2.18 s
├── docs/
│   ├── architecture.svg                     ← system diagram (Figure 1)
│   ├── pipeline.svg                         ← 4-stage validation pipeline (Figure 2)
│   ├── technical_report.md                  ← this report's source
│   ├── technical_report.html                ← rendered HTML intermediate
│   └── technical_report.pdf                 ← rendered PDF submission
└── results/
    ├── eir_translation_three_clauses.ids    ← live Claude output (3 IDS specs)
    ├── generated_demo.ids                   ← live Claude output (1 IDS spec from notebook)
    └── Ifc4_Revit_MEP_remediated.json       ← live Claude output (8 remediation paragraphs)
```

The five regenerable validator reports per IFC (JSON, HTML, BCF, summary CSV, failures CSV) are produced into `results/` at runtime and are gitignored by size policy. The three LLM evidence files in `results/` are exempted from the gitignore and committed.

9. How to reproduce

```
git clone https://github.com/markshanehaines-ZIG/m7u5-ids-mep-validator.git
cd m7u5-ids-mep-validator
python -m venv venv
source venv/Scripts/activate           # Windows Git Bash; venv/bin/activate on macOS/Linux
pip install -r requirements.txt
cp .env.template .env                  # then paste your ANTHROPIC_API_KEY into .env in your editor

# Download the two test models (about 30 seconds on a normal connection)
curl -sL -o ifc/Ifc4_Revit_MEP.ifc \
  https://raw.githubusercontent.com/youshengCode/IfcSampleFiles/main/Ifc4_Revit_MEP.ifc
curl -sL -o ifc/BoilerGasRadiatorDomesticHotWater.ifc \
  "https://raw.githubusercontent.com/EnEff-
BIM/EnEffBIM_UseCases/master/BIM/1.2%20BoilerGasRadiatorDomesticHotWater_VDI%206020/IFC/1.2%20BoilerGasRadiator
DomesticHotWater.ifc"

# Run the validator against the primary IFC
python -m src.cli --ifc ifc/Ifc4_Revit_MEP.ifc --ids ids/mep_services_v1.ids --out results/

# Run the test suite (~2 seconds)
python -m pytest tests/ -v

# Run the LLM modules (requires ANTHROPIC_API_KEY in .env)
python -m src.llm_eir_to_ids --input queries/sample_eir_clauses.txt --out results/generated.ids
python -m src.llm_remediation --report results/Ifc4_Revit_MEP_report.json \
                              --out results/Ifc4_Revit_MEP_remediated.json --limit 20

# Open the demo notebook
jupyter notebook notebooks/01_demo_validation.ipynb

# Rebuild the IDS file (regenerates ids/mep_services_v1.ids from the Python builder)
python scripts/build_mep_ids.py

# Rebuild this PDF (uses Microsoft Edge headless on Windows)
python scripts/build_report_pdf.py
```

The validator's CLI exits 0 on PASS and 1 on FAIL, so a CI job can gate on the exit code directly.

10. Standards referenced

buildingSMART / openBIM

- ISO 16739-1:2024 — Industry Foundation Classes (IFC4)
- buildingSMART IDS 1.0 — Information Delivery Specification (XSD bundled in `ifctester`)
- buildingSMART bSDD — Data Dictionary, at `https://search.bsdd.buildingsmart.org`
- buildingSMART Sample Test Files (CC-BY-4.0)
- BCF 2.1 — BIM Collaboration Format

Information management

- ISO 19650-1 / 19650-2 — Information management using BIM
- ISO 12006-2 — Classification framework
- ISO 55000 — Asset management

MEP services standards cited in the IDS (UK + AS/NZS blend)

- BS EN 12845 §10 — Sprinkler hydraulic calculation (cited in MEP-04 for the assignment; will normalise to NZS 4541 in production)
- BS 7671 §433 — Cable current-carrying capacity (cited in MEP-08 for the assignment; will normalise to AS/NZS 3000 §433)
- AS/NZS 3000 — Wiring rules
- AS/NZS 3500 — Plumbing and drainage
- AS 1668.2 — Mechanical ventilation
- AS 4254 — Ductwork
- NZS 4541 — Automatic fire sprinkler systems
- NZS 4512 — Fire detection and alarm systems

- NZBC C/AS2 — Acceptable solutions for fire safety
- NZBC G4 / G12 / G13 / H1 — Building Code services clauses

Lecture material

- M7U5 Sessions 1–4, Elias Magalhães, Zigurat
- Particular reference to slides 18–24 on IDS as the bridge between EIR intent and machine logic

Source IFC datasets

- `Ifc4_Revit_MEP.ifc` — Autodesk Revit MEP Advanced Sample Project (`rme_advanced_sample_project`), © Autodesk Inc., used here for non-commercial academic evaluation under Autodesk's sample-content terms; IFC4 export republished at `https://github.com/youshengCode/IfcSampleFiles`.
- `BoilerGasRadiatorDomesticHotWater.ifc` — EnEff:BIM project (German Federal Ministry for Economic Affairs and Climate Action research programme), VDI 6020 reference, MIT-licensed at `https://github.com/EnEff-BIM/EnEffBIM_UseCases`.

End of submission.